

Работа с исключениями в Python

sbis.Error

Обработка ошибок.

Пример использования: сервис "Складской учет", модуль "Номенклатура", метод "ClassifierListCached".

```
1 try:
2     classifiers = Классификаторы.СписокПоПолномуКоду(None,
3     Record({'ПолныйКодКлассификатора' : 'ОКЕИ'}), None, None)
4 except Exception as e:
5     raise sbis.Error(e.details, 'Сервис классификаторов временно
6     недоступен.') from e
```

sbis.Warning

Обработка предупреждений.

Пример использования: сервис "Складской учет", модуль "Складской учет", метод "СкладскойДокумент.Провести".

```
1 # Перед проверкой убедимся, что параметры проведения на номенклатурах
2 # документа стоят верные
3 СкладскойДокумент.УстановитьПараметрыПроведения(doc_id,
4 cost_price_mode)
5 orphan_rows = check_orphan_rows(doc_id)
6 if orphan_rows:
7     raise sbis.Warning(
8     user_msg=sbis.rk('Ошибка проведения. {0}'.format( orphan_rows )),
9     details='Ошибка проведения. {0}'.format( orphan_rows )
10 )
11 else:
12     try :
13         СкладскойДокумент.УстановитьЗакрето(doc_id, True)
14     except Exception as e:
15         raise sbis.Warning(e.details, e.user_msg)
16     return True
```

Особенности работы с исключениями

При работе с исключениями в языке Python следует быть осторожным при повторном выбросе исключения. В противном случае можно столкнуться с образованием циклических ссылок и утечками ресурсов (памяти, транзакций, файловых дескрипторов и т.п.) вплоть до следующего вызова garbage collector'a (а он, возможно, будет вызван не скоро).

Рассмотрим следующий фрагмент кода:

```
1 def process_error( ex ):
2     # какая-то логика
3     raise ex
4 def do_job():
5     # какая-то логика
6     try:
7         raise Exception( "foo" )
```

```
8 | except Exception as ex:  
9 |     process_error( ex ) # <--
```

В данном фрагменте происходит обработка исключительной ситуации, вынесенная в отдельную процедуру (process_error).

Данная процедура после выполнения некоторых дополнительных действий кидает исключение дальше. Так как переменная "ex" является локальной по отношению к фрейму функции "process_error", то данный объект (фрейм) имеет на нее ссылку. В точке же генерации исключения к объекту "ex" прикрепляется traceback, который, в свою очередь, имеет ссылку на фрейм. Таким образом, в данном фрагменте образуется циклическая ссылка между объектом исключения и фреймом, в котором он был порожден. Так как каждый фрейм в Python имеет ссылку на предшествующий фрейм, то, в дополнение к указанной паре объектов, "утекают" все фреймы (а значит и все локальные переменные, в них объявленные) ниже по стеку. Данное поведение интерпретатора является [задокументированным](#).

Чтобы не сталкиваться с утечками такого рода, следует избегать повторного выброса объекта-исключения во вложенном фрейме стека.